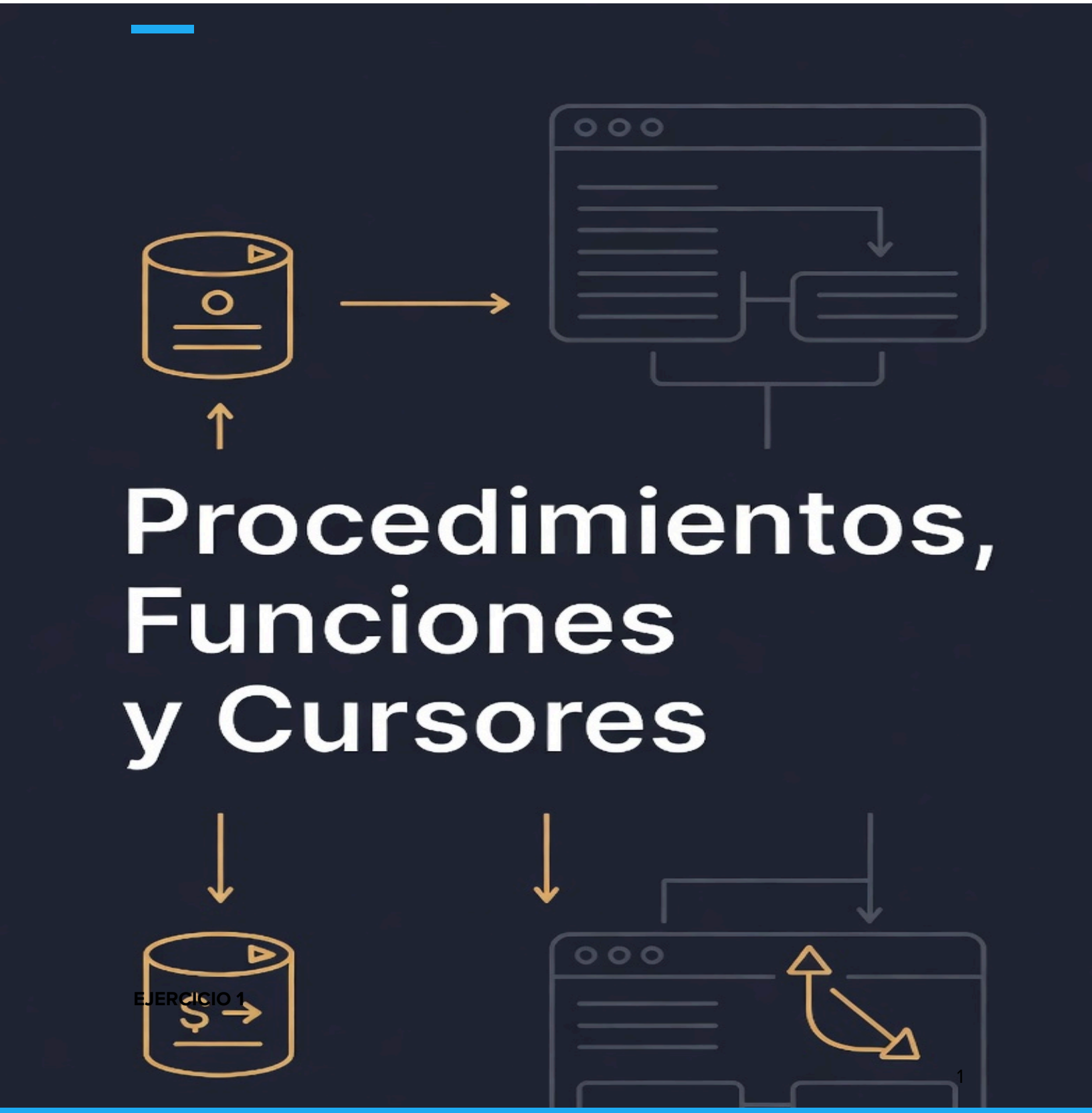
Actividad 1 UT 12 - Procedimientos, Funciones y Cursores

Jose Javier Reyes Hernández



EJERCICIO 1

1.- Vamos a crear un procedimiento almacenado que contenga las siguientes instrucciones:

- Eliminación de la tabla "libros" si existe;

- Creación de la tabla "libros" con: codigo, titulo, autor, editorial, precio, cantidad;

- Ingreso de algunos registros.

Ejecutamos el procedimiento, comprobando que la tabla se ha creado y que los registros son los insertados.

```
create PROCEDURE crearLibros

as


if object_id('libros') is not null

drop table libros;


CREATE TABLE libros(


codigo int PRIMARY KEY,

titulo varchar(20),

autor varchar(20),

editorial varchar(20),

precio DECIMAL(10,2),
```

```
cantidad int



)

 INSERT INTO libros (codigo, titulo, autor,
editorial, precio, cantidad)

   VALUES

       (1, 'Cien Años', 'G. Marquez',
'Sudamericana', 25.50, 10),

       (2, 'El Quijote', 'Cervantes', 'Planeta',
30.00, 5),

       (3, 'La Odisea', 'Homero', 'Gredos', 18.75,
8);


SELECT * from libros

GO


EXEC crearLibros
```

	codigo	titulo	autor	editorial	precio	cantidad
1	1	Cien Años	G. Marquez	Sudamericana	25.50	10
2	2	El Quijote	Cervantes	Planeta	30.00	5
3	3	La Odisea	Homero	Gredos	18.75	8

Finalmente ejecutamos el procedimiento almacenado del sistema "sp_help" y el nombre del procedimiento almacenado para verificar que existe el procedimiento creado recientemente.

```
34  sp_help crearLibros
```

PROBLEMS 2 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	Name	Owner	Type	Created datetime
1	crearLibros	dbo	stored procedure	2026-02-27 14:39:54.697

2.- Necesitamos un procedimiento almacenado que muestre los libros de los cuales hay menos de 10.

```
CREATE PROCedure  librosMenores10

as

SELECT * from libros where cantidad < 10
```

Ejecutamos el procedimiento almacenado del sistema "sp_help" junto al nombre del procedimiento creado recientemente para verificar que existe.

```
43  sp_help librosMenores10
```

PROBLEMS 2 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	Name	Owner	Type	Created datetime
1	librosMenores10	dbo	stored procedure	2026-02-27 14:48:17.617

Lo ejecutamos comprobando que da los registros que solicitamos.

	codigo	titulo	autor	editorial	precio	cantidad
1	2	El Quijote	Cervantes	Planeta	30.00	5
2	3	La Odisea	Homero	Gredos	18.75	8

Modificamos algún registro y volvemos a ejecutar el procedimiento.

(Para ello modifiqué el primer procedimiento para que creara los todos los libros con 11 unidades)

Note que el resultado del procedimiento ha cambiado porque los datos han cambiado.

codigo	titulo	autor	editorial	precio	cantidad

3.- Una empresa almacena los datos de sus empleados en una tabla llamada "empleados".

Cree un procedimiento almacenado llamado "pa_empleados_sueldo" que seleccione los nombres, apellidos y sueldos de los empleados que tengan un sueldo superior o igual al enviado como parámetro.

```
GO

CREATE PROCEDURE pa_empleados_sueldo

@sueldo int

as

SELECT nombre, apellido, sueldo from empleados
```

```
where sueldo >= @sueldo

GO
```

Ejecute el procedimiento creado anteriormente con distintos valores:

exec pa_empleados_sueldo 400;

	nombre	apellido	sueldo
1	Marta	Perez	500.00
2	Susana	Garcia	400.00
3	Jose Maria	Morales	400.00

exec pa_empleados_sueldo 500;

	nombre	apellido	sueldo
1	Marta	Perez	500.00

Ejecute el procedimiento almacenado "pa_empleados_sueldo" sin parámetros.

Mensaje de error.

```
Msg 201, Level 16, State 4, Procedure pa_empleados_sueldo, Line 0
Procedure or function 'pa_empleados_sueldo' expects parameter '@sueldo', which was not supplied.
```

4.- Elimine el procedimiento almacenado "pa_empleados_actualizar_sueldo" si existe:

if object_id('pa_empleados_actualizar_sueldo') is not null

drop procedure pa_empleados_actualizar_sueldo;

Cree un procedimiento almacenado llamado "pa_empleados_actualizar_sueldo" que actualice los sueldos iguales al enviado como primer parámetro con el valor enviado como segundo parámetro.

```
create PROCEDURE pa_empleados_actualizar_sueldo
```

```
@sueldoViejo int ,

@sueldoNuevo int

as

UPDATE empleados set sueldo = @sueldoNuevo

where sueldo=@sueldoViejo

select nombre, apellido, sueldo, cantidadhijos,
seccion from empleados

GO
```

Ejecute el procedimiento creado anteriormente y verifique si se ha ejecutado correctamente:

exec pa_empleados_actualizar_sueldo 300,350;

```
91  exec pa_empleados_actualizar_sueldo 300,350;
```

PROBLEMS 1 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	nombre	apellido	sueldo	cantidadhijos	seccion
1	Juan	Perez	350.00	2	Contaduria
2	Luis	Lopez	350.00	0	Contaduria
3	Marta	Perez	500.00	1	Sistemas
4	Susana	Garcia	400.00	2	Secretaria
5	Jose Maria	Morales	400.00	3	Secretaria

Ejecute el procedimiento "pa_empleados_actualizar_sueldo" enviando un solo parámetro.

Error.

```
Msg 201, Level 16, State 4, Procedure
pa_empleados_actualizar_sueldo, Line 0

Procedure or function
'pa_empleados_actualizar_sueldo' expects parameter
'@sueldoNuevo', which was not supplied
```

Ejecute el procedimiento almacenado "pa_empleados_actualizar_sueldo" enviando en primer lugar el parámetro @sueldonuevo y en segundo lugar @sueldoanterior .

Verifique el cambio:

```
90
91  exec pa_empleados_actualizar_sueldo  350,300
92
```

PROBLEMS 1 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	nombre	apellido	sueldo	cantidadhijos	seccion
1	Juan	Perez	300.00	2	Contaduria
2	Luis	Lopez	300.00	0	Contaduria
3	Marta	Perez	500.00	1	Sistemas
4	Susana	Garcia	400.00	2	Secretaria
5	Jose Maria	Morales	400.00	3	Secretaria

5. -Cree un procedimiento llamado "pa_sueldototal" que reciba el documento de un empleado y muestre su nombre, apellido y el sueldo total (resultado de la suma del sueldo y salario por hijo, que es de $200 si el sueldo es menor a $500 y $100, si el sueldo es mayor o igual a $500).

Coloque como valor por defecto para el parámetro el patrón "%".

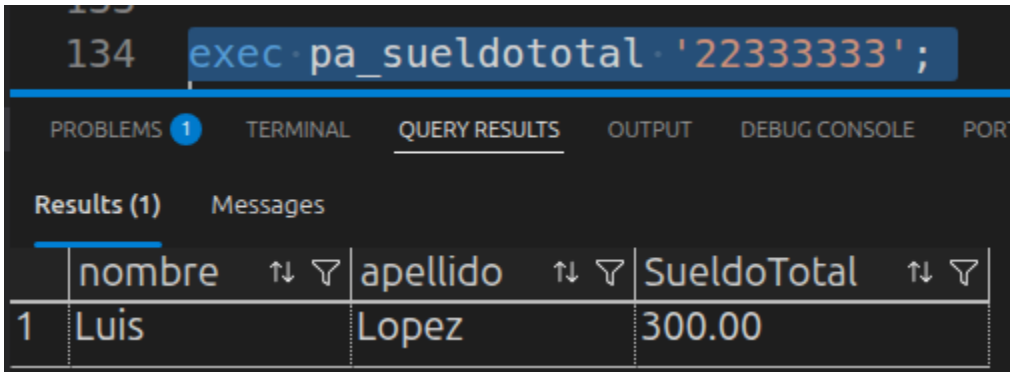
```
create procedure pa_sueldototal
```

```
   @documento char(8)= '%'
AS
IF (SELECT sueldo
FROM empleados
WHERE documento = @documento) < 500
   BEGIN
   SELECT
       nombre,
       apellido,
       sueldo + (cantidadhijos * 200) AS
SueldoTotal
   FROM empleados
   WHERE documento = @documento;
END
   ELSE
   BEGIN
   SELECT
       nombre,
```

```
        apellido,
        sueldo + (cantidadhijos * 100) AS
SueldoTotal
    FROM empleados
    WHERE documento like @documento;
END
GO
```

Ejecute el procedimiento anterior enviando diferentes valores:

exec pa_sueldototal '22333333';



exec pa_sueldototal '22444444';

```
135 exec pa_sueldototal '22444444';
```

PROBLEMS 1 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORT

Results (1) Messages

	nombre	apellido	SueldoTotal
1	Marta	Perez	600.00

exec pa_sueldototal '22666666';

```
136 exec pa_sueldototal '22666666';
```

PROBLEMS 1 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORT

Results (1) Messages

	nombre	apellido	SueldoTotal
1	Jose Maria	Morales	1000.00

EJERCICIO 2

1.- Crear una función que devuelva cuántos libros hay de precio mayor que el que suministremos como parámetro.

```
CREATE PROCEDURE librosMasCaros
@precio DECIMAL(5,2),
@numeroLibros INT OUTPUT
AS
BEGIN
```

```
IF OBJECT_ID('libros') IS NOT NULL
    DROP TABLE libros;

CREATE TABLE libros(
    codigo INT IDENTITY,
    titulo VARCHAR(40),
    autor VARCHAR(30),
    editorial VARCHAR(20),
    precio DECIMAL(5,2)
);

INSERT INTO libros (titulo, autor, editorial,
precio)
VALUES ('Alicia en el pais de las
maravillas','Lewis Carroll','Emece',20.00);

INSERT INTO libros (titulo, autor, editorial,
precio)
```

```
   VALUES ('Alicia en el pais de las
maravillas','Lewis Carroll','Plaza',35.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
   VALUES ('Aprenda PHP','Mario Molina','Siglo
XXI',40.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
   VALUES ('El aleph','Borges','Emece',10.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
   VALUES ('Ilusiones','Richard
Bach','Planeta',15.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
```

```
   VALUES ('Java en 10 minutos','Mario
Molina','Siglo XXI',50.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
   VALUES ('Martin Fierro','Jose
Hernandez','Planeta',20.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
   VALUES ('Martin Fierro','Jose
Hernandez','Emece',30.00);


   INSERT INTO libros (titulo, autor, editorial,
precio)
   VALUES ('Uno','Richard Bach','Planeta',10.00);


   SELECT @numeroLibros = COUNT(*)
   FROM libros
   WHERE precio > @precio;
```

```
END

GO

DECLARE @valor as INTEGER

EXEC librosMasCaros 10,@valor output;

print @valor
```

```
Started executing query at  Line 193
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
7
Total execution time: 00:00:00.056
```

2- Crear una función escalar que tenga como parámetros el DNI y la letra del NIF y nos valide si es correcta o no. Usar la función con los datos de una tabla que contenga nombre, apellidos, fechanacimiento, dni y la letra del nif.

"La letra del DNI español se calcula a partir de los 8 números del DNI. Primero se toma ese número y se hace una división entre 23, quedándose solo con el resto de la división. Ese resto será un número entre 0 y 22, que se usa como posición dentro de una cadena fija de letras: "TRWAGMYFPDXBNJZSQVHLCKE". Cada posición de esa cadena corresponde a una letra. "

Sabiendo eso lo usamos para crear la función

```
CREATE FUNCTION validarDNI
```

```
(
    @dni CHAR(8),
    @Nif CHAR(1)
)
RETURNS VARCHAR(10)
AS
BEGIN
    DECLARE @letras CHAR(23) =
'TRWAGMYFPDXBNJZSQVHLCKE'
    DECLARE @posicion INT
    DECLARE @letraCalculada CHAR(1)

    SET @posicion = CAST(@dni AS INT) % 23
    SET @letraCalculada = SUBSTRING(@letras,
@posicion + 1, 1);

    IF @letraCalculada = @Nif
        RETURN 'correcta'
```

```
    RETURN 'incorrecta'

END
```

```
238  SELECT
239      nombre, apellidos,  dbo.validarDNI(dni, letra) AS resultado
240  FROM personas;
```

PROBLEMS 4 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	nombre	apellidos	resultado
1	Juan	Pérez	correcta
2	María	Hernández	incorrecta
3	Ana	Rodríguez	correcta

3.- Crear una función que nos devuelva los años de diferencia respecto al actual a partir de la fecha pasada como parámetro. Usar la función con la tabla anterior.

```
CREATE FUNCTION aniosDiferencia

(

@fechaInput DATETIME

)

RETURNs int

as

BEGIN
```

```
DECLARE

@fechaActual DATETIME,

@diferencia int

SET @fechaActual = GETDATE()

SET @diferencia = DATEDIFF(YEAR, @fechaInput,
@fechaActual)RETURN @diferencia

end
```

```
260 SELECT
261     nombre, apellidos,  dbo.aniosDiferencia(fechanacimiento)
262 FROM personas;
```

PROBLEMS 5 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	nombre	apellidos	(No column name)
1	Juan	Pérez	56
2	María	Hernández	41
3	Ana	Rodríguez	35

4.- Crear función que dada fecha como cadena de caracteres devuelva que no es correcta o en caso contrario el nombre del mes.

Para esta función usamos trycast para castear el input como si fuera una fecha y al ser trycast no dara error al programa porque es como si tuviera un try catch

```
CREATE FUNCTION nombreMes

(
```

```
    @fechaInput VARCHAR(30)
)
RETURNS VARCHAR(30)
AS
BEGIN
    DECLARE @resultado VARCHAR(30)

    DECLARE @fecha DATE
    SET @fecha = TRY_CONVERT(DATE, @fechaInput)

    IF @fecha IS NULL
    SET @resultado = 'formato incorrecto'
    ELSE
    SET @resultado = DATENAME(MONTH, @fecha)

    RETURN @resultado
END
GO
```

```
284  SELECT dbo.nombreMes('2025-03-15')
285  SELECT dbo.nombreMes('abc')
```

PROBLEMS 6 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (2) Messages

	(No column name)
1	March

	(No column name)
1	formato incorrecto

5.- Crear una función de tabla que devuelva los libros de precio mayor que el que suministremos como parámetro.

```
CREATE FUNCTION  librosMasCarosFunc
(
   @precio DECIMAL(10,2)
)
RETURNS TABLE
as
```

```
RETURN(SELECT  titulo,precio FROM libros WHERE
precio>@precio)
```

```
300   SELECT * FROM librosMasCarosFunc(20)
```

PROBLEMS 6 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	titulo	precio
1	Alicia en el pais de las maravillas	35.00
2	Aprenda PHP	40.00
3	Java en 10 minutos	50.00
4	Martin Fierro	30.00

6.- Función que devuelva el máximo precio de la tabla libros

Se usa un select con top 1 y order by para obtener el precio mayor y asignarlo a la variable

```
CREATE FUNCTION libroMasCaro()

RETURNS DECIMAL(10,2)

AS

BEGIN

   DECLARE @precioMax DECIMAL(10,2)



   SELECT TOP 1 @precioMax = precio
```

```
    FROM libros

    ORDER BY precio DESC;


    RETURN @precioMax

END
```

```
315   SELECT dbo.libroMasCaro() AS PrecioMasAlto;
```

PROBLEMS 6 TERMINAL QUERY RESULTS OUTPUT DEBUG CONSOLE PORTS

Results (1) Messages

	PrecioMasAlto
1	50.00

7.- Función que devuelva una tabla con el nombre de las personas de dos tablas (personal y alumnado), pasando como parámetro: personal (saca sólo los de la tabla personal; alumnado (saca sólo los de la tabla alumnado; ambos (saca los de ambas tablas)

```
CREATE FUNCTION  ObtenerPersonas (@tipo VARCHAR(20))
RETURNS TABLE

AS

RETURN

(
```

```
    SELECT Nombre FROM personal WHERE @tipo IN ('personal', 'ambos')
    UNION ALL
    SELECT Nombre FROM alumnado WHERE @tipo IN ('alumnado', 'ambos')
)
GO
```

8.- Función que devuelva el nº de días del mes de una fecha pasada como parámetro.

La función mira el mes y devuelve el número de días ,a no ser que sea febrero en el cual cogeremos el año para saber si es bisiesto antes de devolver el resultado

```
CREATE FUNCTION DiasDelMes (@fecha DATE)
RETURNS INT
AS
BEGIN
    DECLARE @mes INT
    DECLARE @anio INT

    SET @mes = MONTH(@fecha)
    SET @anio = YEAR(@fecha)

    IF @mes = 1 OR @mes = 3 OR @mes = 5 OR @mes = 7
       OR @mes = 8 OR @mes = 10 OR @mes = 12
        RETURN 31

    IF @mes = 4 OR @mes = 6 OR @mes = 9 OR @mes = 11
```

```
        RETURN 30




    IF (@anio % 4 = 0 AND @anio % 100 <> 0) OR (@anio % 400 = 0)

        RETURN 29

    ELSE

        RETURN 28

END

GO
```

9.- Crear función que valide si una cadena de caracteres es un DNI correcto. Que contenga 8 dígitos y una letra y la letra se corresponda con la correcta. Probarlo con la tabla personas.

En sql no existen booleanos sino que usamos BIT para valores de true o false asi que usaremos eso como retorno y en la funcion usaremos cosas que ya vimos en las anteriores como calcular la letra del dni y el trycast

```
CREATE FUNCTION ValidarDNI (@dni VARCHAR(9))

RETURNS BIT

AS

BEGIN

    DECLARE @numero INT

    DECLARE @letra CHAR(1)

    DECLARE @letraCorrecta CHAR(1)

    DECLARE @cadena VARCHAR(23) = 'TRWAGMYFPDXBNJZSQVHLCKE'
```

```
    IF LEN(@dni) <> 9
        RETURN 0

    SET @numero = TRY_CAST(LEFT(@dni,8) AS INT)
    SET @letra = UPPER(RIGHT(@dni,1))

    IF @numero IS NULL
        RETURN 0

    SET @letraCorrecta = SUBSTRING(@cadena, (@numero % 23) + 1, 1)

    IF @letra = @letraCorrecta
        RETURN 1
    ELSE
        RETURN 0
END
GO
```

EJERCICIO 3

Crear un procedimiento almacenado que recorra una tabla con un cursor. Esta tabla contendrá en un campo el nombre de la tabla a crear y en otros dos las descripciones de los dos campos que compondrán dicha tabla crear. El procedimiento deberá crear todas las tablas que se indiquen en TablasCrear.

```
CREATE PROCEDURE CrearTablasDesdeConfiguracion
AS
BEGIN
  DECLARE
       @Tabla NVARCHAR(100),
       @DescripcionCampo1 NVARCHAR(100),
       @DescripcionCampo2 NVARCHAR(100),
       @SQL NVARCHAR(MAX)

   DECLARE cursor_tablas CURSOR FOR
   SELECT Tabla, DescripcionCampo1,
DescripcionCampo2
   FROM TablasCrear

   OPEN cursor_tablas

   FETCH NEXT FROM cursor_tablas
   INTO @Tabla, @DescripcionCampo1,
@DescripcionCampo2
```

```
    WHILE @@FETCH_STATUS = 0
    BEGIN

        SET @SQL = '

        BEGIN
            CREATE TABLE ' +  @Tabla + ' (
                ' + @DescripcionCampo1 + ',
                ' + @DescripcionCampo2 + '
            )
        END'

        EXEC sp_executesql @SQL

        FETCH NEXT FROM cursor_tablas
        INTO @Tabla, @DescripcionCampo1,
@DescripcionCampo2
```

```
    END



    CLOSE cursor_tablas

    DEALLOCATE cursor_tablas

END

GO


EXEC CrearTablasDesdeConfiguracion
```